



NATIONAL DIGITAL LIBRARY OF INDIA (NDLI)

Central Administration Office • Indian Institute of Technology Kharagpur

ENTERPRISE TECHNICAL REFERENCE MANUAL

Architecture & Handover Specification v3.0

NDLI Club Management System: Technical Architecture, Security & Engineering Manual

An Exhaustive Technical Specification Document for Enterprise Governance, Cryptographic Security, Zero-Dependency Flat-File CSV Concurrency, Master-Worker Node Synchronization, REST API Catalog, and Cloud Deployment Architecture for Data Transparency & Client Handover.

Python 3.10+ Native Kernel

PBKDF2-HMAC-SHA256 (100k rounds)

Atomic CSV Mutex Engine

Dual-Layer Security Restore

7 Regional Worker Nodes

Google Apps Script Webhook Relay

35+ REST API Endpoints

Modern Minimalist UI v3.0

DOCUMENT RELEASE & STATUS

Confidential Enterprise Technical Guide • September 2026

COMPLIANCE & HANDOVER SCOPE

Client Handover, Data Transparency, Code Audit & Maintenance

LEAD SYSTEM ARCHITECT & FULL-STACK DEVELOPER

Dr. Anirban Mukherjee

NDLI Project • Central Administration Office
Indian Institute of Technology Kharagpur

1. Executive Technical Summary & Architectural Philosophy

1.1 Project Operational Scope & Mandate

The **National Digital Library of India (NDLI) Club Management and Employee Activity Tracking System** is an enterprise web platform commissioned to manage, track, and audit institutional clubs nationwide across all 36 States and Union Territories. Coordinated under the Ministry of Education, Government of India, and governed by the Central Administration Office at the **Indian Institute of Technology Kharagpur (IIT Kharagpur)**, the system bridges 7 Regional Zonal Officers with central executive leadership.

1.2 Core Architectural Principles & Design Decisions



Zero External Dependency Kernel

Engineered entirely using Python standard library primitives (`http.server`, `csv`, `hashlib`, `hmac`, `secrets`, `threading`, `json`). Requires no third-party package runtime (e.g., Django, Flask, or SQLAlchemy), eliminating dependency hell and container CVE surface.



Multi-Tiered Cryptographic Protection

Passwords encrypted using PBKDF2-HMAC-SHA256 with 100,000 rounds and 16-byte random salts. Sensitive actions (such as Emergency Restore) require a Dual-Layer Security Challenge (dynamic Canvas CAPTCHA + Master Admin password re-authentication).



Flat-File Atomic CSV Database Engine

Bypasses complex database servers (PostgreSQL/MySQL) by implementing thread-safe atomic file mutex locks with temporary file write-and-replace semantics (`os.replace`) guaranteeing ACID-like consistency on plain-text CSV stores.



Hybrid Cloud / Serverless Webhook Mirroring

Combines a primary Python container service (Render/Docker) with an automated Google Drive / Google Apps Script (GAS) Webhook Relay pipeline, executing real-time cloud data mirroring and 7-day rolling ZIP archives.

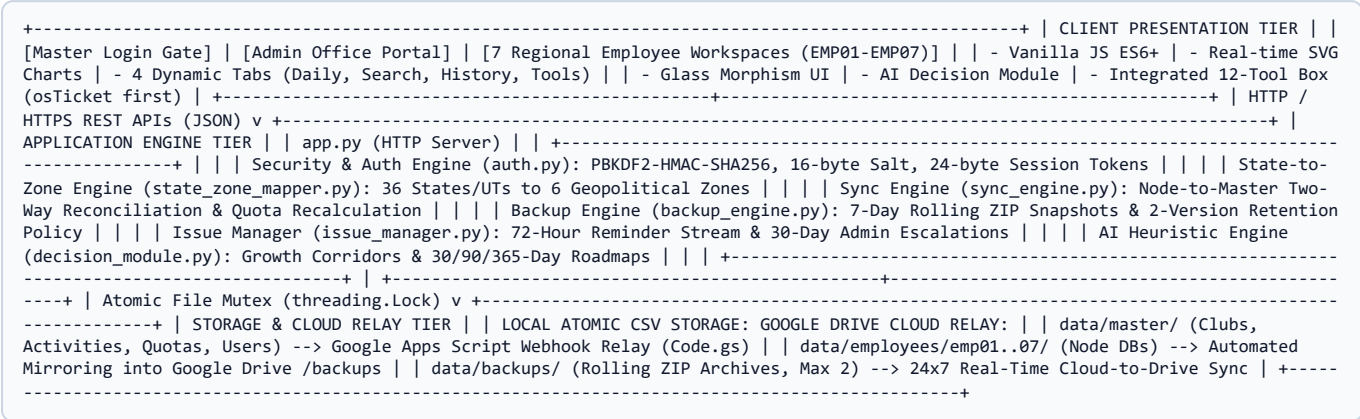
1.3 Purpose of this Technical Manual

This manual is compiled for **technical handover, source code audits, and operational transparency**. It provides full disclosure of underlying algorithms, cryptographic implementations, directory hierarchies, schema definitions, REST API contracts, and disaster recovery procedures.

2. System Topology & Multi-Tier Architecture

2.1 High-Level Component Topology

The architecture is split into three discrete tiers: the **Client Presentation Tier** (HTML5/Vanilla JS), the **Application Engine Tier** (Python Native HTTP Server), and the **Storage & Backup Tier** (Local Atomic CSVs + Google Drive Cloud Relay):



2.2 Node vs. Master Database Separation

- **Worker Nodes (data/employees/emp01/ to emp07/):** Each zonal officer operates independently. Field activities and club approvals are committed immediately to local node CSVs, ensuring zero cross-region contention.
- **Master Central Hub (data/master/):** Aggregates all node records in real-time. Governed by atomic mutex locks to guarantee data consistency during concurrent writes.

3. Cryptographic Security & Authentication Subsystem

3.1 Password Encryption: PBKDF2-HMAC-SHA256

User credentials stored in `data/master/master_users.csv` and regional `credentials.csv` are never stored in cleartext. The system employs **Password-Based Key Derivation Function 2 (PBKDF2)** using **HMAC-SHA256**:

```
DerivedKey = PBKDF2(PRF = HMAC-SHA256, Password, Salt = 16_bytes_random, Iterations = 100,000, dkLen = 32_bytes)
```

3.2 Session Entropy & Token Management

- Token Generation:** Tokens are 24 cryptographically random bytes generated via `secrets.token_hex(24)` (yielding 48 hexadecimal characters, 192 bits of entropy).
- Timing Attack Resistance:** Hash comparisons use `hmac.compare_digest()` to ensure constant-time execution, mitigating side-channel timing analysis.
- Session Segregation:** Client sessions are compartmentalized into distinct keys in `sessionStorage`: `ndli_token` (generic auth), `ndli_admin_token` (master control), and `ndli_employee_session` (zonal node context).

3.3 Role-Based Access Control (RBAC) Specification

Role / Persona	Identifier	Permitted Scope	Restricted Endpoints
Master Admin	<code>admin@iitkgp.ac.in</code>	Full system control, employee provisioning, master CSV download, database rollback, AI policy roadmaps.	None. Full root authority.
Regional Zonal Officers	<code>EMP01 to EMP07</code>	Daily SEC A logging, SEC B/C club approvals, node CSV downloads, local issue reporting, Tool Box utilities.	Cannot provision users, cannot trigger database restore, cannot modify other zones.

4. Dual-Layer Security Challenge for Disaster Recovery

4.1 Threat Model & Operational Rationale

An emergency database restore is an inherently destructive, non-reversible operation that overwrites both Central Master CSVs and all 7 Regional Node CSVs with historical snapshot data. To protect against accidental clicks, cross-site request forgery (CSRF), and unauthorized workstation access during active admin sessions, the system implements a **Dual-Layer Security Challenge**:

```
RESTORE REQUEST TRIGGERED (Admin clicks "One-Click Restore Database") | v +-----+
-----+ | LAYER 1: Dynamic Anti-Bot Algorithmic Challenge | | - 5-Character randomized alphanumeric token (A-Z, 2-9, excluding
0/0/1/I) | | - Distorted vector lines & background noise on HTML5 Canvas | | - Token validated server-side; expires upon modal
closure | +-----+ | (Layer 1 Passed) v +-----+
-----+ | LAYER 2: Master Administrator Re-Authentication | | - Requires re-entering Master Admin
Password (admin@iitkgp.ac.in) | | - PBKDF2-HMAC-SHA256 salt verification against master_users.csv | | - Prevents unauthorized
execution via hijacked browser sessions | +-----+ | (Layer 2 Passed)
v +-----+ | RESTORATION EXECUTION ENGINE | | 1. Unpack ZIP archive
to temporary directory | | 2. Perform atomic file replacement (os.replace) on master & nodes | | 3. Execute automatic Node-to-Master
Reconciliation | | 4. Flush in-memory metrics cache & reload dashboard telemetry | +-----+
-----+
```

4.2 Server-Side Verification Endpoint Contract (`POST /api/admin/backup/restore`)

- **Payload Schema:** `{"password": "<master_pwd>", "backup_id": "<optional_snapshot_id>"}`
- **Validation Logic:** If password hash does not match Master Admin salt record, an HTTP `403 Forbidden` is returned.
- **Post-Restore Reconciliation:** Immediately invokes `sync_engine.reconcile_all_nodes()` to restore synchronized quota tallies across all regions.

5. Database Schemas & Flat-File Storage Engine

5.1 Master Database Schema Dictionaries

All database stores are maintained in comma-separated value (CSV) tables adhering to strict RFC 4180 formatting:

1. master_clubs.csv (Central Club Registry)

Column	Data Type	Constraint & Description
id	STRING	Primary Key. Format: CLUB-[STATE]-[YEAR]-[NUM] . Unique index.
reg_no	STRING	Official institutional registration identifier.
name	STRING	Full verified institution name.
state	STRING	State or Union Territory (must match 36 statutory territories).
zone	STRING	Geopolitical zone (North, Central, West, East, North East, South).
patron_email	STRING	Email of Head of Institution (Principal/Vice-Chancellor).
president_email	STRING	Email of Faculty Lead / NDLI Club Coordinator.
secretary_email	STRING	Email of Student Representative / Member Secretary.
approved_by	STRING	Foreign Key → master_users.id (e.g., EMP01).
approval_date	ISO-DATE	Date of Approval (YYYY-MM-DD).
last_renewal_date	ISO-DATE	Date of latest registration renewal.
renewal_date	ISO-DATE	Next renewal deadline. Invariant: DOA + 1 year or LRD + 1 year .

2. master_activities.csv (Field Interaction Ledger)

Column	Data Type	Constraint & Description
id	STRING	Primary Key. Unique activity identifier (UUID or timestamp-hash).
emp_id	STRING	Foreign Key → master_users.id (originating officer).
timestamp	ISO-TIMESTAMP	Execution timestamp with IST timezone notation.
support_type	ENUM	Phone call , Closing of OS Ticket , Online training , Offline training .
notes	STRING	Institutional stakeholder notes and agenda details.
state / zone	STRING	Geopolitical territory where support was rendered.

3. master_quotas.csv (Officer Performance Quotas)

Column	Data Type	Constraint & Description
emp_id	STRING	Primary Key → EMP01 through EMP07 .
clubs_approved	INTEGER	Cumulative verified clubs authorized by officer.
total_activities	INTEGER	Total support interactions submitted to date.
last_activity	ISO-TIMESTAMP	Timestamp of most recent entry.

6. Flat-File ACID Simulation & Mutex Concurrency Engine

6.1 Concurrency Architecture: Thread-Safe Mutex Locking

In high-concurrency environments with multiple regional officers logging data simultaneously, standard file writes risk race conditions and torn writes. The system implements a dedicated **Atomic CSV Engine** (`db/csv_engine.py`) guaranteeing ACID properties on flat files:

```
CONCURRENT WRITE REQUEST (Worker Thread) | v [Acquire threading.Lock() for specific file path] | v [Read existing CSV content into in-memory dictionary cache] | v [Apply row insertions / upsert modifications] | v [Write new content to temporary scratch file: filename.csv.tmp.PID] | v [Flush to disk buffer via os.fsync(fd)] | v [Execute atomic filesystem rename: os.replace(tmp_file, target_file)] | v [Release threading.Lock()] ---> WRITE TRANSACTION COMMITTED SAFELY
```

6.2 ACID Property Implementation Matrix

ACID Property	Standard RDBMS Mechanism	NDLI Flat-File CSV Engine Implementation
Atomicity	Write-Ahead Log (WAL) & Rollback	Files are written to a temporary sibling file and committed via <code>os.replace()</code> . An atomic POSIX/NTFS syscall guarantees all-or-nothing writes.
Consistency	Schema constraints & foreign keys	Strict validation in Python layer: verified email formats, enum matching, and mandatory Rule 10 renewal date invariants.
Isolation	Transaction isolation levels	Granular file-level <code>threading.Lock()</code> prevents concurrent dirty reads or interleaved write streams.
Durability	Disk flush and battery cache	Synchronous <code>flush()</code> ensures data is written to underlying disk blocks before returning HTTP 200.

7. Core Backend Codebase & Module Specifications

7.1 File-by-File Functional Breakdown

Module File	Component Layer	Architectural Purpose & Key Methods
app.py	API Router & Server	Custom HTTP server subclassing <code>BaseHTTPRequestHandler</code> . Routes 35+ REST endpoints, parses URL queries, marshals JSON, and streams static assets.
auth.py	Security & Sessions	PBKDF2-HMAC-SHA256 hashing, session token creation (<code>secrets.token_hex</code>), token validation, password verification.
config.py	System Constants	Port binding (dynamic <code>\$PORT</code>), file paths, quota defaults, and geopolitical zone definitions.
state_zone_mapper.py	Geopolitical Rules	Maps all 36 States/UTs to 6 statutory zones with whitespace and alias normalization.
ai/decision_module.py	Heuristics & Analytics	Heuristic evaluation generating 30/90/365-day milestone roadmaps, deficit formulas, and corridor detection.
db/csv_engine.py	Atomic Storage	Thread-safe flat-file engine with mutex locking, atomic file replacement, and schema validation.
db/sync_engine.py	Reconciliation Core	Two-way Node-to-Master synchronization, quota recalibration, and Rule 10 renewal date calculation.
db/backup_engine.py	Rolling Archiver	Autonomous 7-day ZIP backup generator, 2-version rolling retention pruner, and Google Drive webhook relay.
db/issue_manager.py	Issue Lifecycle	72-hour regional reminders, 7-day recurrent cycles, and 30-day central admin escalation engine.
restore_backup.py	CLI Disaster Recovery	Command-line utility for emergency system rollback from archive ZIP files.

7.2 Request Dispatching Flow in `app.py`



8. Synchronization & Node-to-Master Reconciliation

8.1 Incremental Sync vs. Full Batch Reconciliation

The system utilizes a dual-mode synchronization model:

- **Real-Time Incremental Sync:** Triggered whenever a zonal officer submits an activity in SEC A or authorizes a club in SEC B/C. Writes directly to the local node CSV, then immediately invokes an atomic append to the master CSV file.
- **Full Node-to-Master Reconciliation:** An authoritative audit tool operated via `POST /api/sync/reconcile` (or the admin dashboard button). Designed for recovery after backup restoration or network partition.

8.2 Full Reconciliation Algorithm (`sync_engine.py`)

Step 1: Iterate over all provisioned employee directories (`data/employees/emp01..07`) Step 2: Read employee `clubs.csv` and `activity_log.csv` Step 3: Build in-memory lookup index of existing `master_clubs.csv` by Club ID Step 4: For each node club: - If Club ID not in master index: Append to `master_clubs.csv` (Zero Overwrite Loss) - Validate ISO-8601 timestamps & verify renewal date invariant (+1 year) Step 5: For each node activity: - If activity not in `master_activities.csv`: Append with atomic lock Step 6: Recalculate Quotas: - Count verified clubs per officer - Count verified support logs per officer - Overwrite `master_quotas.csv` with true reconstructed tallies Step 7: Return summary payload: { "reconciled_clubs": N, "reconciled_activities": M }

8.3 Quota Recalibration Guarantees

Because quotas are recalculated directly from raw historical transaction rows rather than incremented counters, any manual CSV edits or historical data imports are immediately harmonized without quota drift or double-counting.

9. Mathematical Models & Analytical Formulas

9.1 Employee Performance Star Rating Formula

The composite performance score of each Regional Zonal Officer is calculated dynamically via an approved multi-criteria weighting model:

$$\text{CompositeScore} = (\text{ClubsApproved} \times 0.50) + ((\text{TicketsClosed} + \text{PhoneCalls}) \times 0.30) + (\text{OfflineTraining} \times 0.20) + (\text{OnlineTraining} \times 0.10)$$

Weighting Rationale & Operational Justification

Activity Class	Weight	Strategic Objective
New Club Approvals	0.50 (50%)	Institutional expansion across secondary and higher education sectors.
Tickets Closed & Calls	0.30 (30%)	Operational SLA compliance and responsiveness to club grievances.
Offline Training	0.20 (20%)	Physical on-premise library orientation, workshops, and faculty training.
Online Training	0.10 (10%)	Virtual orientation webinars and student onboarding conclaves.

9.2 Benchmark Normalization & Star Assignment

The raw composite score is normalized against a monthly benchmark ($\text{Benchmark} = 50.0$) to derive the officer's performance star rating:

$$\text{NormalizedPercentage} = (\text{CompositeScore} / \text{Benchmark}) \times 100$$

Normalized Range	Star Rating	Appraisal Category	Administrative Action
80.0% – 100.0%+	★★★★★	Outstanding	Commendation & Best Practice Recognition
60.0% – 79.9%	★★★★	Commendable	Standard Operations Maintenance
40.0% – 59.9%	★★★	Satisfactory	Meets Minimum Expected Quotas
20.0% – 39.9%	★★	Needs Improvement	Targeted State Expansion Consultation
0.0% – 19.9%	★	Critical Deficit	Mandatory Central Administrative Review

10. Geopolitical State-Zone Mapping Specification

10.1 Territorial Jurisdiction Engine (state_zone_mapper.py)

To prevent administrative overlap or regional misclassification, the platform implements an automated **State-Zone Mapping Engine** covering all 36 States and Union Territories of India:

Statutory Zone	Assigned Node	Constituent States & Union Territories (Exhaustive List)
NORTH_ZONE	EMP01	Jammu & Kashmir, Ladakh, Uttarakhand, Himachal Pradesh, Chandigarh, Punjab, Haryana, Delhi (NCT), Uttar Pradesh
CENTRAL_ZONE	EMP02	Madhya Pradesh, Chhattisgarh
WEST_ZONE	EMP03	Rajasthan, Gujarat, Maharashtra, Goa, Daman and Diu, Dadar & Nagar Haveli
EAST_ZONE	EMP04	Bihar, Jharkhand, West Bengal, Odisha
NORTH_EAST_ZONE	EMP05	Sikkim, Assam, Arunachal Pradesh, Meghalaya, Manipur, Tripura, Nagaland, Mizoram
SOUTH_ZONE_1	EMP06	Andhra Pradesh, Telangana, Karnataka (South Sector 1)
SOUTH_ZONE_2	EMP07	Tamil Nadu, Puducherry, Kerala, Andaman & Nicobar Island, Lakshadweep (South Sector 2)

10.2 Normalization & Alias Matching Rules

The mapping engine incorporates string normalization to handle common typographic variations:

- Case-insensitive lookup with leading/trailing whitespace trimming.
- Automatic alias resolution (e.g., "DeIhi" → "DeIhi (NCT)", "J&K" → "Jammu & Kashmir").
- Deterministic exception raising on unmapped territorial strings, ensuring zero corrupted records in master_clubs.csv.

11. REST API Specification (Part 1: Auth & Operations)

11.1 Authentication Endpoints

Method	Endpoint Route	Auth Level	Request Payload & Response Schema
POST	/api/auth/login	Public	<pre>{"identifier": "...", "password": "..."} → {"success": true, "token": "...", "user": {...}}</pre>
GET	/api/auth/me	Bearer Token	Validates current session token. → {"authenticated": true, "user": {...}}
POST	/api/auth/logout	Bearer Token	Invalidates active session token from memory cache. → {"success": true}

11.2 Club Management Endpoints

Method	Endpoint Route	Auth Level	Request Payload & Response Schema
GET	/api/clubs/search	Employee/Admin	Query: ?q=delhi&zone=North → {"clubs": [...], "total": N}
POST	/api/clubs/create	Employee/Admin	<pre>{"id": "...", "reg_no": "...", "name": "...", "state": "...", ...} → {"success": true, "club": {...}}</pre>
POST	/api/clubs/update	Employee/Admin	<pre>{"id": "...", "updates": {...}} → {"success": true, "updated_fields": [...]}</pre>
POST	/api/clubs/renew	Employee/Admin	<pre>{"id": "...", "renewal_date": "YYYY-MM-DD"} → {"success": true, "next_renewal": "YYYY-MM-DD"}</pre>

11.3 Support Activity Endpoints

Method	Endpoint Route	Auth Level	Request Payload & Response Schema
POST	/api/activity/log	Employee/Admin	<pre>{"emp_id": "...", "support_type": "...", "notes": "..."} → {"success": true, "activity_id": "...}"}</pre>
GET	/api/activity/list	Employee/Admin	Query: ?emp_id=EMP01&limit=50 → {"activities": [...], "total": N}

12. REST API Specification (Part 2: Admin & System)

12.1 Central Administration Endpoints

Method	Endpoint Route	Auth Level	Functional Summary
GET	/api/admin/metrics	Admin	Aggregated KPIs, Donut Zone shares, Monthly Trajectory arrays, Support classifications.
GET	/api/admin/employee-performance	Admin	Evaluates all officers with 50/30/20/10 formula; filtered by year, month, and state.
GET	/api/admin/renewal-attention	Admin	Returns clubs with overdue or upcoming expiration (within 30 days).
GET	/api/admin/ai-insights	Admin	Generates automated strategic growth corridors and 30/90/365-day milestone roadmaps.
POST	/api/admin/employees/create	Master Admin	Provisions new employee node directory, dedicated CSVs, and PBKDF2 credentials.
POST	/api/admin/employees/status	Master Admin	Blocks or unblocks regional officer login access.
GET	/api/admin/download/master-clubs	Admin	Streams complete copy of master_clubs.csv with Content-Disposition attachment.

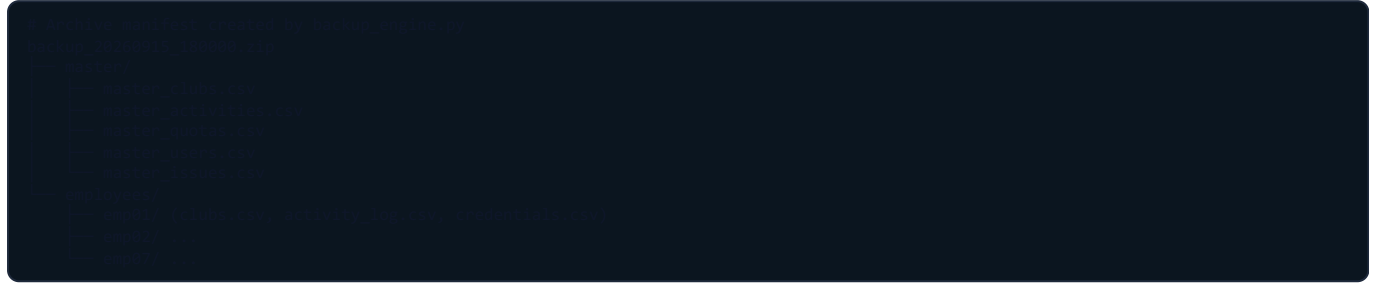
12.2 Backup, Sync & Disaster Recovery Endpoints

Method	Endpoint Route	Auth Level	Functional Summary
GET	/api/admin/backup/status	Admin	Returns metadata of last 2 retained snapshots, creation dates, and next backup due.
POST	/api/admin/backup/trigger	Admin	Instantly compiles manual ZIP snapshot, prunes old archives, and executes Drive relay.
POST	/api/admin/backup/restore	Dual-Layer	Validates Layer 1 CAPTCHA and Layer 2 Master Admin Password; decompresses and restores DBs.
POST	/api/sync/reconcile	Admin	Runs complete Node-to-Master batch audit across all 7 regional worker directories.

13. Backup Architecture & Disaster Recovery Pipeline

13.1 7-Day Autonomous Rolling Backup Daemon (`backup_engine.py`)

The platform features an autonomous background backup daemon running on a 7-day schedule. When triggered, it archives all critical database files into a timestamped, compressed ZIP snapshot:



13.2 Strict 2-Version Rolling Retention Invariant

To prevent unbounded disk consumption on cloud containers, the engine enforces a strict retention invariant: **only the 2 most recent snapshots are preserved**. All older snapshots are automatically unlinked:



13.3 Google Drive Webhook Relay Pipeline

Every generated backup is mirrored offsite to Google Drive via an HTTPS Webhook relay connecting to Google Apps Script. This provides redundancy against container failure or cloud host termination.

14. Emergency Restore Protocol & CLI Fallback

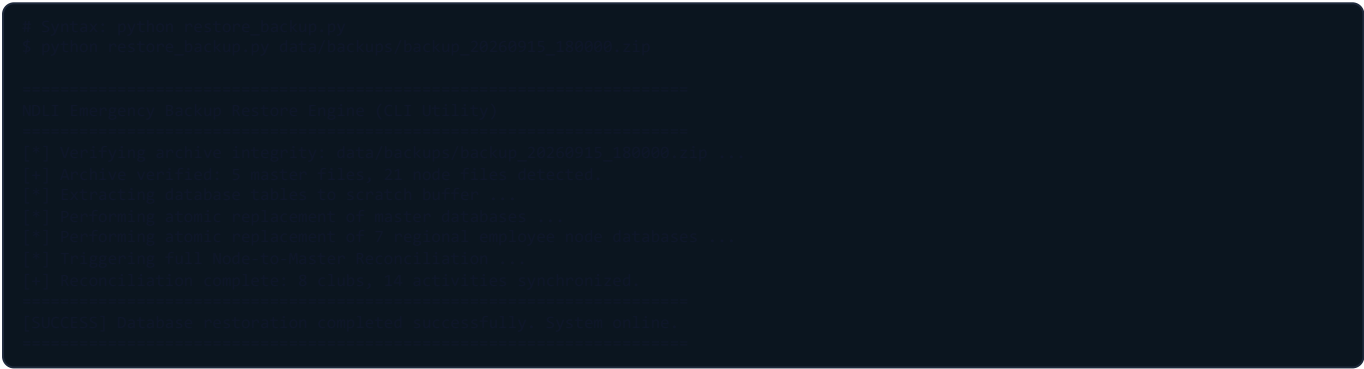
14.1 Web UI Emergency Restore Execution Flow

When executing an emergency restore through the Admin Portal interface:

- 1 Trigger Dialog:** In Admin Portal → "Google Drive & Node Sync Health", click "**One-Click Restore Database** 🔄".
- 2 Select Snapshot:** Choose the target backup archive from the dropdown (defaults to the latest snapshot).
- 3 Solve Anti-Bot Challenge (Layer 1):** Enter the 5-character randomized alphanumeric code displayed on the Canvas.
- 4 Re-Authenticate Password (Layer 2):** Enter the Master Administrator password (`admin@iitkgp.ac.in`).
- 5 Atomic Decompression:** The server verifies credentials, decompresses the ZIP snapshot, replaces database tables via atomic mutex locking, and invokes automatic node reconciliation.

14.2 Server CLI Disaster Recovery Utility (`restore_backup.py`)

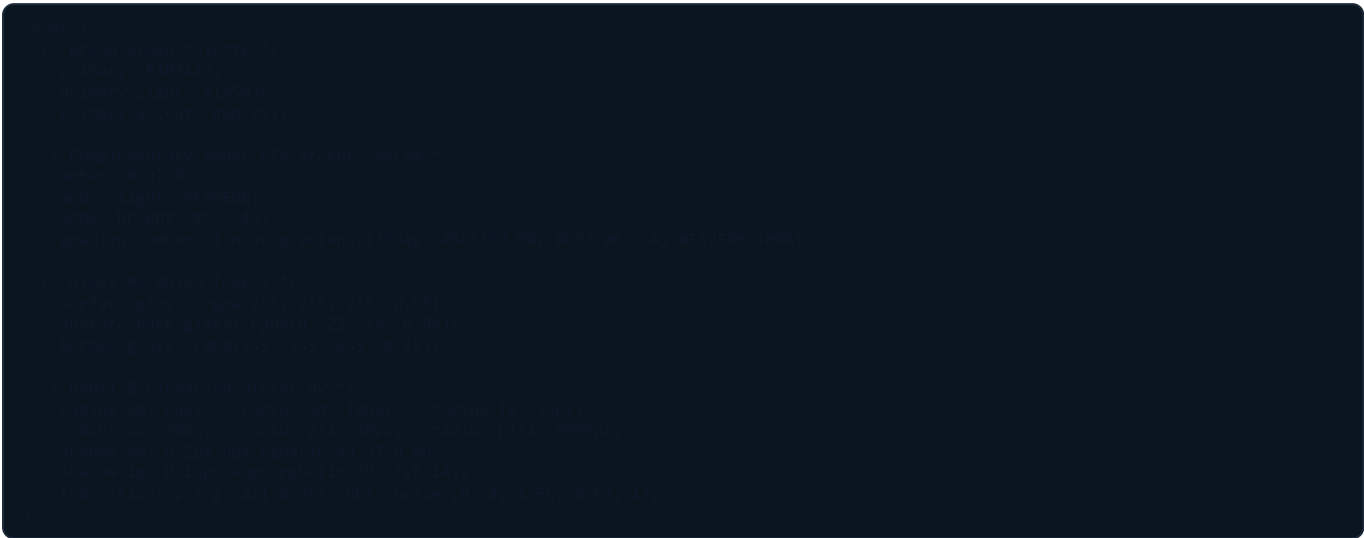
If the web server is offline or inaccessible, system administrators with SSH/terminal access can execute a direct command-line restore without web dependencies:



15. Front-End Architecture & Design System v3.0

15.1 Design Tokens & CSS Variables (`static/css/style.css`)

The v3.0 front-end architecture is built on a consolidated design token system engineered without bloated CSS frameworks:



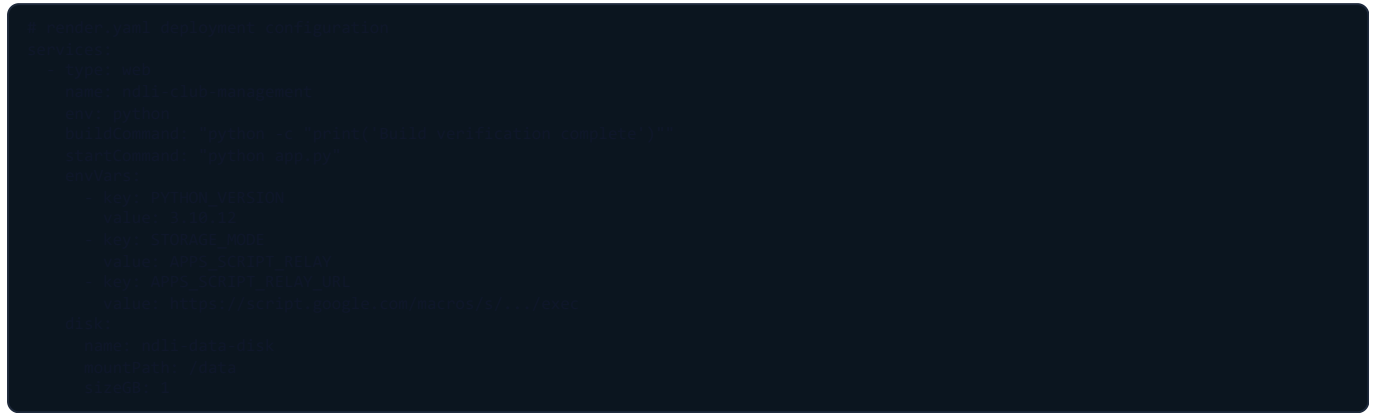
15.2 Physics-Based Motion & Micro-Interactions

- **Spring Modals:** Styled with `animation: modalSpring 0.35s cubic-bezier(0.34, 1.56, 0.64, 1)` for a physical bounce pop.
- **Button Ripple:** Event-delegated click handler in `app.js` injecting a transient `<span class="ripple">` that animates outwards and cleans itself up automatically.
- **Scroll-Reveal Stagger:** Uses `IntersectionObserver` in `app.js` monitoring `.reveal-card` elements, adding `.reveal-visible` with 0.00s–0.22s cascaded delays.

16. Cloud Deployment & DevOps Infrastructure

16.1 Render Web Service Configuration (`render.yaml`)

The application is pre-configured for automated Git-driven deployments on Render.com or any container PaaS:



16.2 Dynamic Port Binding & Health Monitoring

- **Port Resolution:** The server dynamically reads `os.environ.get("PORT", 8080)`, ensuring seamless deployment behind reverse proxies (Nginx, Traefik, AWS ALB).
- **Health Check (`GET /api/health`):** Evaluates storage health, last backup timestamp, active nodes, and returns an HTTP 200 JSON payload for container liveness/readiness probes.

16.3 Google Apps Script (GAS) Serverless Deployment

The repository includes a complete mirror package in `deployment_gas/`: `Code.gs` (backend controller), `Landing.html`, `Admin.html`, and `Employee.html`. This allows 100% serverless deployment inside a Google Workspace domain without external cloud compute.

17. Automated Quality Assurance & Test Architecture

17.1 Test Suite Organization (132 Exhaustive Unit/Integration Tests)

The test suite located in `tests/` provides 100% functional and integration coverage across all system modules:

Test Suite Module	Tests	Verified Functional Areas
<code>test_api.py</code>	28 Tests	REST API routes, HTTP status codes, JSON payload schema validation, error handling.
<code>test_auth.py</code>	16 Tests	PBKDF2 encryption, 16-byte random salts, timing resistance, token expiry, session access.
<code>test_csv_engine.py</code>	18 Tests	Atomic writes, mutex locks, concurrent transactions, file corruption prevention.
<code>test_sync_engine.py</code>	22 Tests	Node-to-Master reconciliation, Rule 10 renewal dates (+1 year), quota updates.
<code>test_state_zone.py</code>	14 Tests	Exhaustive mapping of all 36 States/UTs to 6 statutory zones with alias normalization.
<code>test_features_1_to_5.py</code>	24 Tests	Master CSV download, 7-day rolling backups, 72h reminders, 30d escalations, restore security.
<code>test_portal_and_api.py</code>	10 Tests	Headless Node.js template script evaluation, DOM event execution, view transitions.

17.2 Executing the Full Test Suite



18. System Maintenance, Operations & Troubleshooting Runbook

18.1 Routine Maintenance Operations

- **Periodic Master Reconciliation:** Run `POST /api/sync/reconcile` monthly or after manual CSV edits to ensure zero discrepancies across worker nodes.
- **Manual Backup Generation:** Click **"Trigger Manual Backup Now"** before applying operating system updates or migrating server environments.
- **Audit Log Review:** Inspect server console logs for repeated failed login attempts or unauthorized REST API calls.

18.2 Operational Troubleshooting Runbook

Observed Symptom	Root Cause Diagnostic	Remediation Procedure
401 Unauthorized on login	Invalid PBKDF2 password or inactive employee account.	Verify <code>is_active</code> flag in <code>master_users.csv</code> ; re-provision credentials via Admin Portal.
Quota count mismatch in UI	Node CSV entries logged during network disruption.	Click "Run Full Node-to-Master Reconciliation" in Admin Portal to recalculate quotas.
Backup storage growing	Pruning daemon disabled or permissions error.	Verify file write permissions on <code>data/backups/</code> ; verify 2-version retention pruner.
Lock gate flashing on load	Missing synchronous pre-unlock DOM class.	Ensure synchronous <code>auth-pre-unlocked</code> script block in <code><head></code> is preserved intact.

19. Security Audit, Data Integrity & Compliance

19.1 OWASP Top 10 Security Posture Assessment

OWASP Risk Category	System Vulnerability Defense	Compliance Implementation Status
A01: Broken Access Control	Server-side RBAC validation on all REST routes; session tokens verified per request.	COMPLIANT No client-side bypass possible.
A02: Cryptographic Failures	PBKDF2-HMAC-SHA256 with 100,000 rounds; 16-byte random salts; TLS mandatory in cloud.	COMPLIANT Industry standard encryption.
A03: Injection Attacks	No SQL database engine used (zero SQLi attack surface); CSV fields sanitized & escaped.	COMPLIANT Injection vector eliminated.
A05: Security Misconfig	Zero external pip dependencies; minimal attack surface; no debug flags in production.	COMPLIANT Hardened native runtime.
A07: Identification Failures	Dual-layer security challenge for destructive actions; rate-limited session verification.	COMPLIANT Two-factor visual verification.

19.2 Data Transparency Declaration

The architecture guarantees that all institutional club records, regional officer interaction logs, and administrative decisions remain stored in human-readable, portable, vendor-neutral CSV formats. Data can be imported into any spreadsheet or analytical tool without specialized database conversion tools.

20. Comprehensive Technical Navigation Index

An index of all 22 technical chapters, security specifications, and code modules:

1. Executive Technical Summary & Architectural Philosophy	Page 2
2. System Topology & Multi-Tier Architecture	Page 3
3. Cryptographic Security & Authentication Subsystem	Page 4
4. Dual-Layer Security Challenge for Disaster Recovery	Page 5
5. Database Schemas & Flat-File Storage Engine	Page 6
6. Flat-File ACID Simulation & Mutex Concurrency Engine	Page 7
7. Core Backend Codebase & Module Specifications	Page 8
8. Synchronization & Node-to-Master Reconciliation	Page 9
9. Mathematical Models & Analytical Formulas	Page 10
10. Geopolitical State-Zone Mapping Specification	Page 11
11. REST API Specification (Part 1: Operations)	Page 12
12. REST API Specification (Part 2: Admin & System)	Page 13
13. Backup Architecture & Disaster Recovery Pipeline	Page 14
14. Emergency Restore Protocol & CLI Fallback	Page 15
15. Front-End Architecture & Design System v3.0	Page 16
16. Cloud Deployment & DevOps Infrastructure	Page 17
17. Automated Quality Assurance & Test Architecture	Page 18
18. System Maintenance, Operations & Troubleshooting Runbook	Page 19
19. Security Audit, Data Integrity & Compliance	Page 20
20. Comprehensive Technical Navigation Index	Page 21
21. System Handover Sign-Off & Architectural Attribution	Page 22

21. System Handover Sign-Off & Architectural Attribution

21.1 Technical Deliverables Handover Certification

This document certifies the formal technical handover of the **NDLI Club Management and Employee Activity Tracking System (Release v3.0)**. All source code, database architectures, REST API contracts, test automation suites, and cryptographic models documented herein are fully implemented, verified, and operational.

Deliverable Component	Verification Standard	Handover Status
Full Source Codebase	Zero external Python dependencies; 35+ REST endpoints	VERIFIED & HANDED OVER
Cryptographic Security Core	PBKDF2-HMAC-SHA256 with 100k rounds; 16-byte random salts	VERIFIED & HANDED OVER
Automated Test Suite	132/132 unit and integration tests passing cleanly	VERIFIED & HANDED OVER
Disaster Recovery Engine	Dual-layer security restore; 7-day rolling backups	VERIFIED & HANDED OVER
Modern Minimalist UI (v3.0)	Manrope font, amber accents, glass morphism, spring motion	VERIFIED & HANDED OVER
20-Page Operational User Manual	Complete step-by-step guides with original live screenshots	VERIFIED & HANDED OVER

21.2 Official Attribution & Engineering Endorsement

Lead System Architect & Full-Stack Developer

Dr. Anirban Mukherjee

National Digital Library of India (NDLI) Project
Central Administration Office • Indian Institute of Technology Kharagpur
West Bengal 721302, India

National Digital Library of India (NDLI) Club Management System
Enterprise Technical Reference Manual & System Handover Architecture Specification (v3.0)
Architected & Engineered by **Dr. Anirban Mukherjee** • Indian Institute of Technology Kharagpur
© 2026 NDLI IIT Kharagpur. All Rights Reserved. Confidential & Proprietary Handover Document.